TrustNav — oral examination: speaker notes

Group 2, MSc Cybersecurity and Data Science, ESAIP Angers · Thursday 13 November 2026, 09:35, room C201 · Tutor: Sofiane Hamrioui

Running order (about 15 minutes, then questions)

Slide	Speaker	Time	Message in one line
1 Title	Karan	1 min	One question: can a fleet believe its satellite position?
2 Problem	Sudheer	1.5 min	Real incidents; nothing today scores, explains and acts per vehicle.
3 System	Karan	1.5 min	A software layer beside the receiver: scores, explains, responds.
4 Witnesses	Karan	2 min	Rules, anomaly model, OSNMA; fused by noisy-OR, not averaged.
5 Response	Karan	1.5 min	Accept, hold, release; nothing downstream consumes a lie.
6 Evaluation	Niranjan	2 min	Disjoint routes; 99.3 % precision, 96.1 % recall, 0.44 % false alarms, with intervals.
7 Timelines	Niranjan	1.5 min	7/7 episodes, median 1 s, 8 km reduced to 29 m, no blank screen.
8 Real receiver	Niranjan	1.5 min	Rules transfer as they are; the model needs re-calibration; two fixes.
9 Demo	Karan + Pavan	3 min	Replay the teleport, the map, the phone as a receiver.
10 Claims	Pavan	1.5 min	Limits, ethics, next steps; one closing sentence.

Before the room

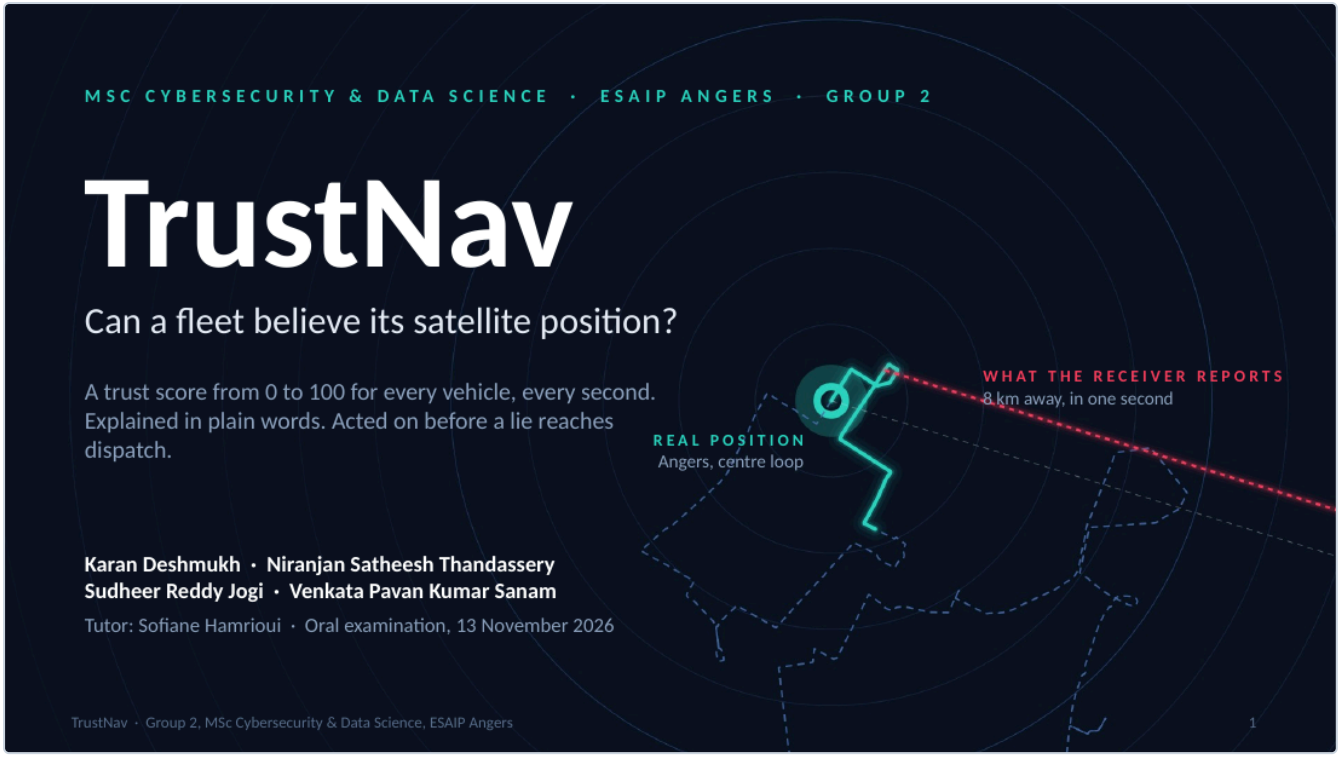
Open **trustnav.vercel.app** in a browser tab before the talk; load Replay → *Teleport spoof* and pause it before the attack (press **P** for presentation mode if the room screen is small).

Phone: open **trustnav.vercel.app/phone**, token pasted, outdoors or near a window for a fix; start the laptop engine (`python -m trustnav_engine.phone`) with the token in `engine/.env.local`. If no fix indoors, show the attack switch on the recorded replay instead.

Fallbacks if the network fails: the PDF of this deck, the paper's Fig. 3 and Fig. 4, and the screenshots on slides 5 and 9. Say so in one sentence and move on.

Numbers to keep in mind: 36 features, 15 rules, 10 held-out scenarios, 3 800 observations (1 550 attacked, 2 250 clean), 32 tests, 109 end-to-end checks, 27.6-minute real drive.

1 TrustNav: can a fleet believe its satellite position?



Speaker: Karan (1 min)

Good morning. We are Group 2 and this is TrustNav. One question sums up the project: can a fleet believe the satellite position its vehicles report?

Every dispatch screen, geofence and flight controller today assumes the answer is yes. TrustNav is a software layer that checks that assumption once a second, for every vehicle and drone, gives each position a trust score from 0 to 100, explains the score in plain words, and acts on it when the position cannot be trusted.

In the next fifteen minutes: why the problem is real, how TrustNav decides, how we evaluated it, what a real receiver taught us, and a live demonstration. We will keep the demo short and leave time for your questions.

2 Everything that moves trusts GNSS blindly



Speaker: Sudheer (1.5 min)

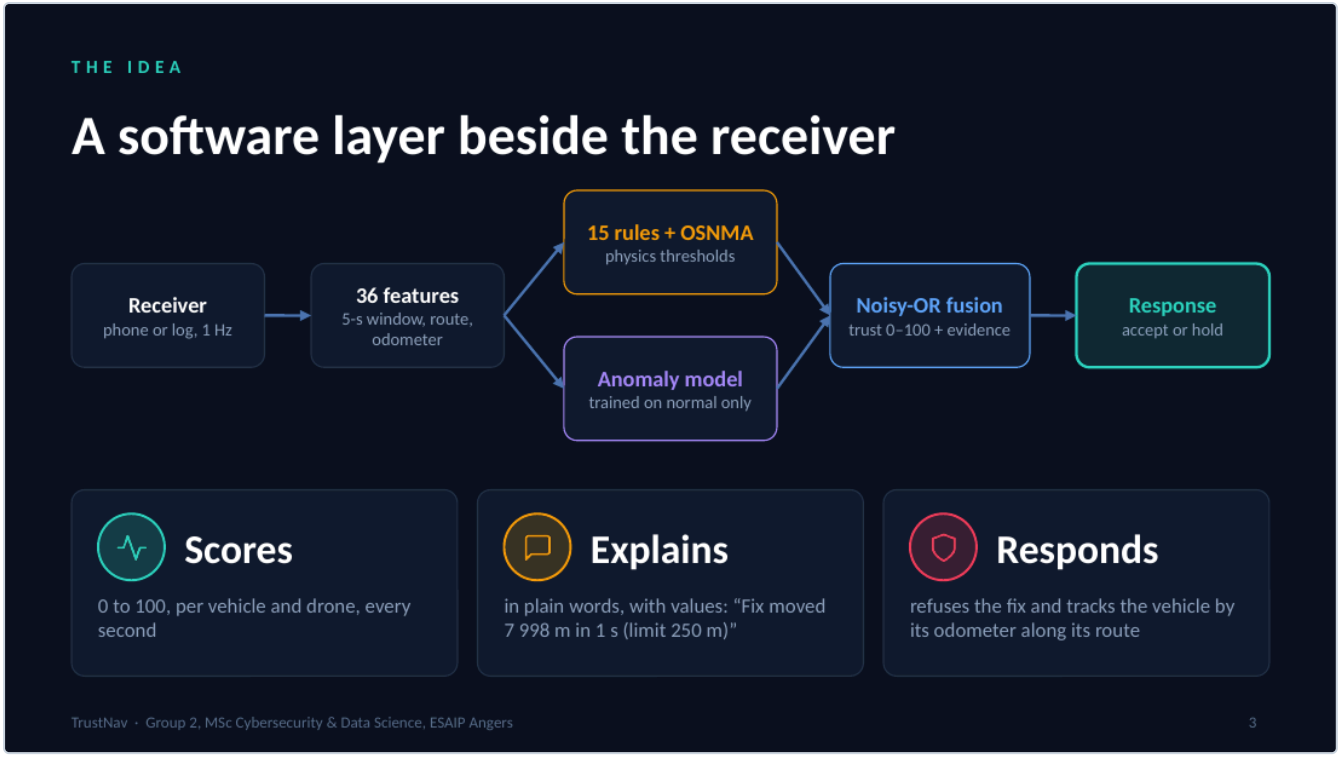
Why does this matter? GNSS signals arrive from 20 000 km away and are extremely weak, so a small transmitter nearby can drown them, which is jamming, or imitate them and feed a believable false position, which is spoofing.

This is not theoretical and not far away. In April 2017 a jammer left running in a car parked at Nantes airport delayed several flights; the owner was fined two thousand euros. In July 2023 a truck driver near Lyon used a jammer to escape his company's tracking and disrupted the airport until the ANFR traced him. And since 2022 interference has been a daily event for aviation around the Baltic and conflict zones, about fifteen hundred flights a day spoofed in the summer of 2024.

What exists today does not close the gap. Fleet telematics simply displays the position it is given; at best it relays a jamming flag. Receiver manufacturers like u-blox or Septentrio do detect inside the receiver, but they output a state code with no explanation, and at Jammertest 2025 both the jamming and the spoofing flags tripped during jamming alone. Fixed probes and anti-jamming antennas protect a site, a port or an airport; they do not travel with a van.

So the gap is this: nobody tells the operator, per vehicle and per second, whether the position is true, and nobody acts on it. That is what TrustNav does.

3 A software layer beside the receiver



Speaker: Karan (1.5 min)

TrustNav sits beside the receiver and works only on what any receiver already reports: position, speed, satellites, signal strength, dilution of precision, clock, and where available the Galileo authentication status and the vehicle's own odometer speed. The source can be a real receiver, a phone, or a recorded log, once per second.

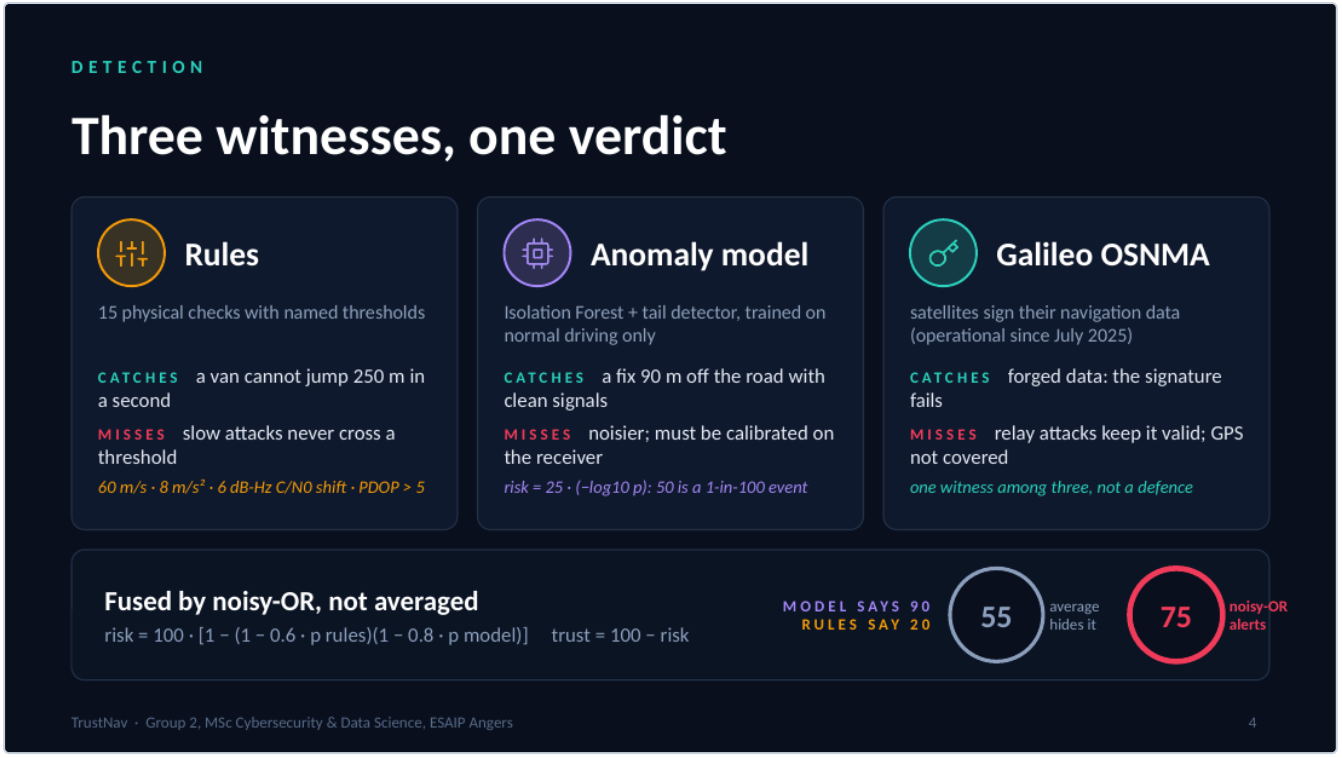
From each second and the five before it, we compute 36 features: how far the fix moved, derived versus reported speed, distance to the planned route, changes in signal strength, satellite count, clock, and the mismatch between GNSS speed and odometer speed. Absent inputs are flagged explicitly, never mistaken for zero.

Two independent witnesses then judge the features: fifteen physical rules with named thresholds, and an anomaly model trained on normal driving only. Their risks are fused by noisy-OR into one trust score between 0 and 100, with the evidence attached.

The score drives a response: an accepted fix goes downstream unchanged; a refused fix is replaced by an estimate. Everything is also sent to the console and the incident log.

Three promises follow: TrustNav scores, every second; it explains, in plain words with the actual values; and it responds, so a lie never reaches dispatch, geofencing or flight control.

4 Three witnesses, one verdict



Speaker: Karan (2 min)

Let me open the three witnesses.

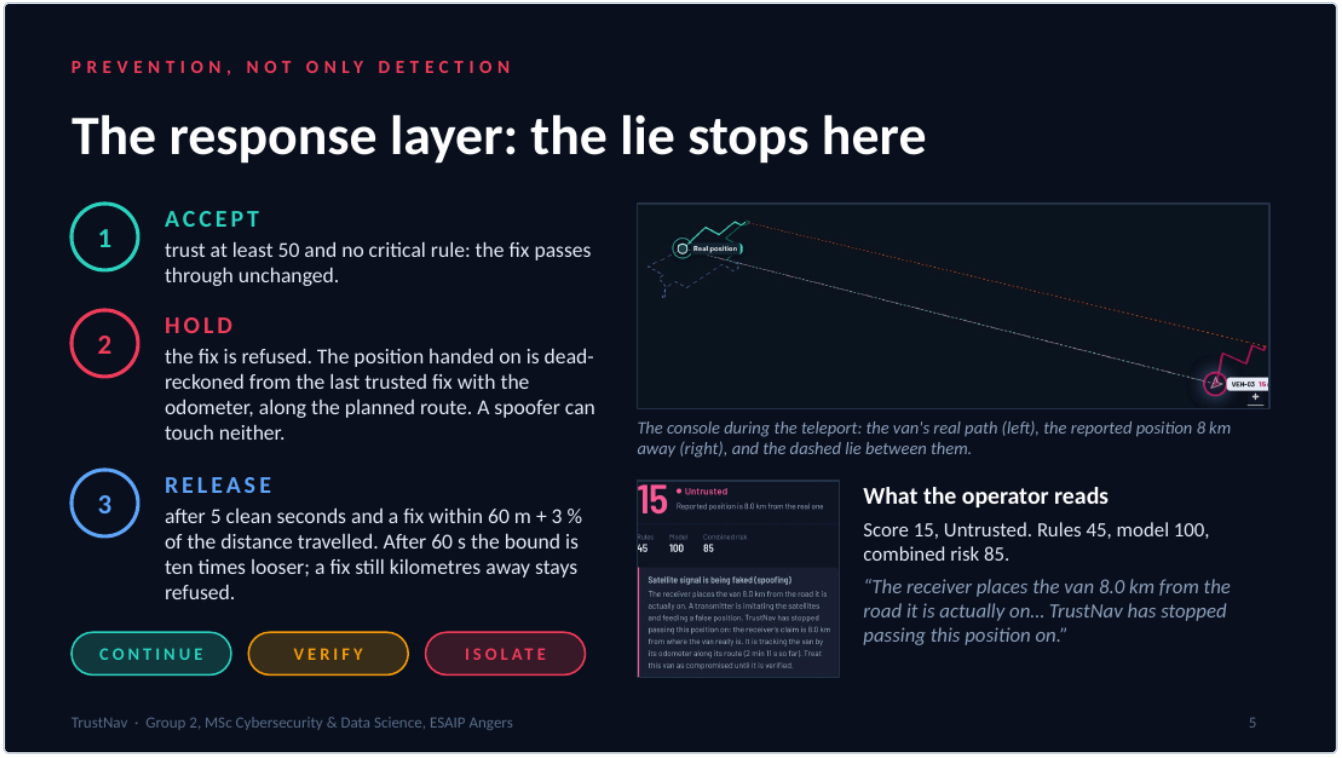
The rules are fifteen deterministic checks against named thresholds. A road vehicle cannot exceed 60 metres per second, accelerate at more than 8 metres per second squared, or jump 250 metres in one second. On the signal side, a 6 dB-Hz shift in carrier-to-noise ratio, a PDOP above 5, a clock-drift change above 60 nanoseconds per second. Two conditions are critical and pin the risk at 70 or more: a jump of four times the limit, and a failed OSNMA signature. The rules are almost silent on normal data, and that is also their weakness: a slow attack never crosses a threshold.

The anomaly model covers that blind spot. It is an Isolation Forest with 300 trees plus a per-feature tail detector, and it is trained on normal operation only, never on an attack, so it cannot memorise the simulator's attacks. Its raw score is converted to a calibrated tail probability on normal runs it has not seen, and the risk is 25 times minus log10 of that probability. A risk of 50 therefore means a one-in-a-hundred event under normal driving. The tail detector also names the odd input: a fix 90 metres off the road while the signals are clean.

Galileo OSNMA is the third witness: the satellites sign their navigation data, operational since July 2025. A failed signature means forged data and is critical. A valid signature is not proof of safety: a relay attack re-broadcasts genuine signals, and GPS is not covered at all.

The witnesses are combined by noisy-OR, not averaged. If the model says 90 and the rules say 20, an average would give 55 and hide the alarm; noisy-OR gives 75. The two weights were chosen by grid search on validation routes only.

5 The response layer: the lie stops here



Speaker: Karan (1.5 min)

Detection alone leaves the operator looking at a lie on the screen. The response layer turns the score into an action, the way an intrusion prevention system acts on what an intrusion detection system finds.

A fix is accepted when trust is at least 50 and no critical rule fired. Otherwise it is held: the fix is refused, and the position handed to downstream software is dead-reckoned from the last trusted fix using the odometer speed, advanced along the planned route. A spoofer can fake satellites; it cannot touch the odometer or the route.

The hold is released only after five clean seconds and a returning fix that rejoins the estimate within 60 metres plus 3 percent of the distance travelled, so a spoofer cannot end the hold by simply looking clean while still far away. After sixty clean seconds the tolerance is loosened ten times, because an odometer estimate drifts too, but a fix still kilometres away stays refused.

Every second also carries an action for the operator: continue, verify by other means, or isolate the device when trust is below 25 or a critical rule fired.

On the right is what the operator sees: the score, the band, the three risks and the verdict in plain words, and on the map the real position, the protected position on the road, and the reported lie eight kilometres away.

6 Tested on routes it has never seen



Speaker: Niranjan (2 min)

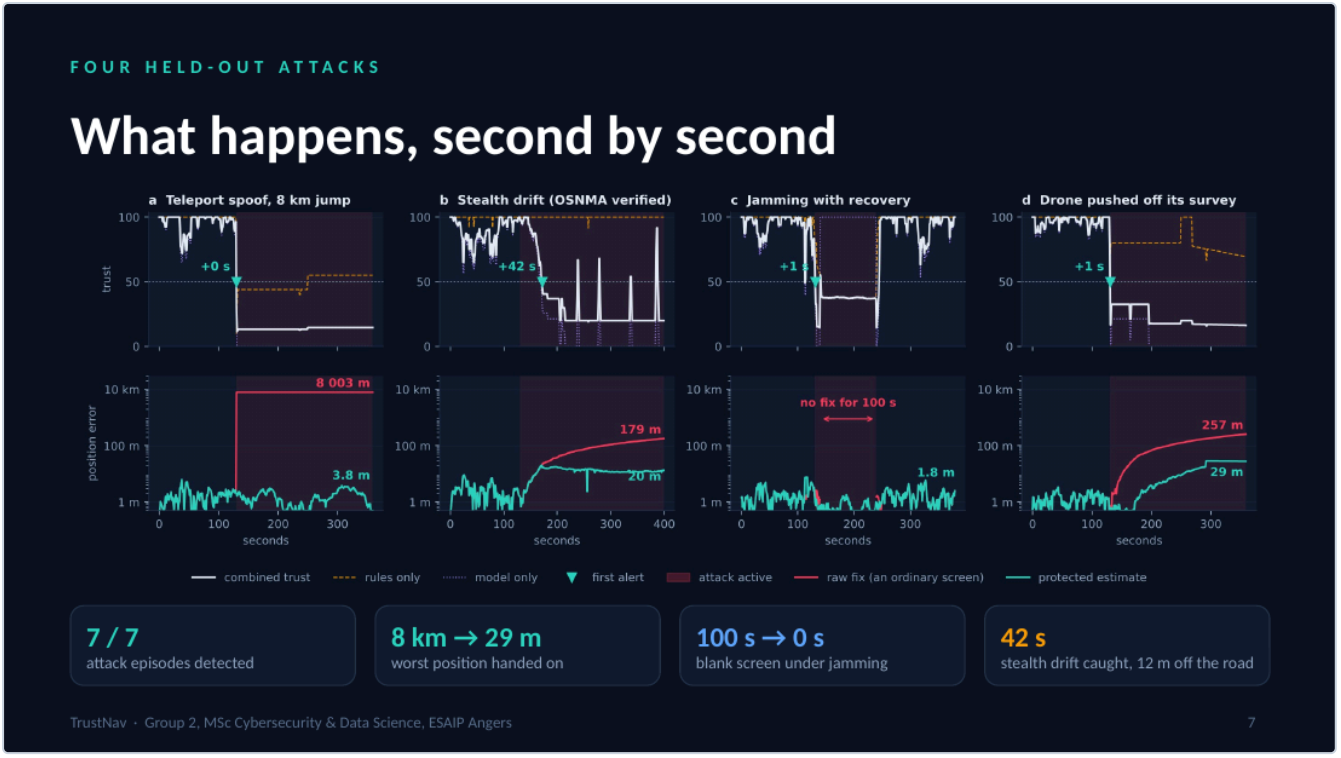
How did we test it? No public dataset gives labelled per-second receiver observables for fleets with routes and odometers, and transmitting attacks is illegal, so a deterministic, seeded simulator produces what a receiver would report, over real Angers street geometry.

The important discipline is that every route has exactly one role. Five road loops and two drone areas train the model, with normal runs only. Two loops and one drone area are validation, and the only thing chosen on them is the pair of fusion weights. Two loops and one drone area are held out and produce every number we report: ten scenarios, 3 800 one-second observations, 1 550 of them under attack, seven attack types. Rule thresholds come from physics, not from data, and an alert is a combined risk of 50 or more.

The chart compares four ways to decide. The baseline is the classic signal check, carrier-to-noise below 35 dB-Hz or fewer than five satellites: perfect precision but 9 percent recall, because it only sees jamming. The rules alone never raise a false alarm but miss slow attacks, 28 percent recall. The model alone reaches 90 percent recall but is noisier, 1.33 percent false alarms. Combined, each covers the other's blind spot: 99.3 percent precision, 96.1 percent recall, F1 0.977, 0.44 percent false alarms.

Because seconds within a run are correlated, we also bootstrapped over scenarios: the 95 percent intervals are 97.9 to 100 for precision and 89.4 to 99.6 for recall. At the event level, all seven attack episodes are detected, the median delay is one second, and no false incident is ever opened.

7 What happens, second by second



Speaker: Niranjan (1.5 min)

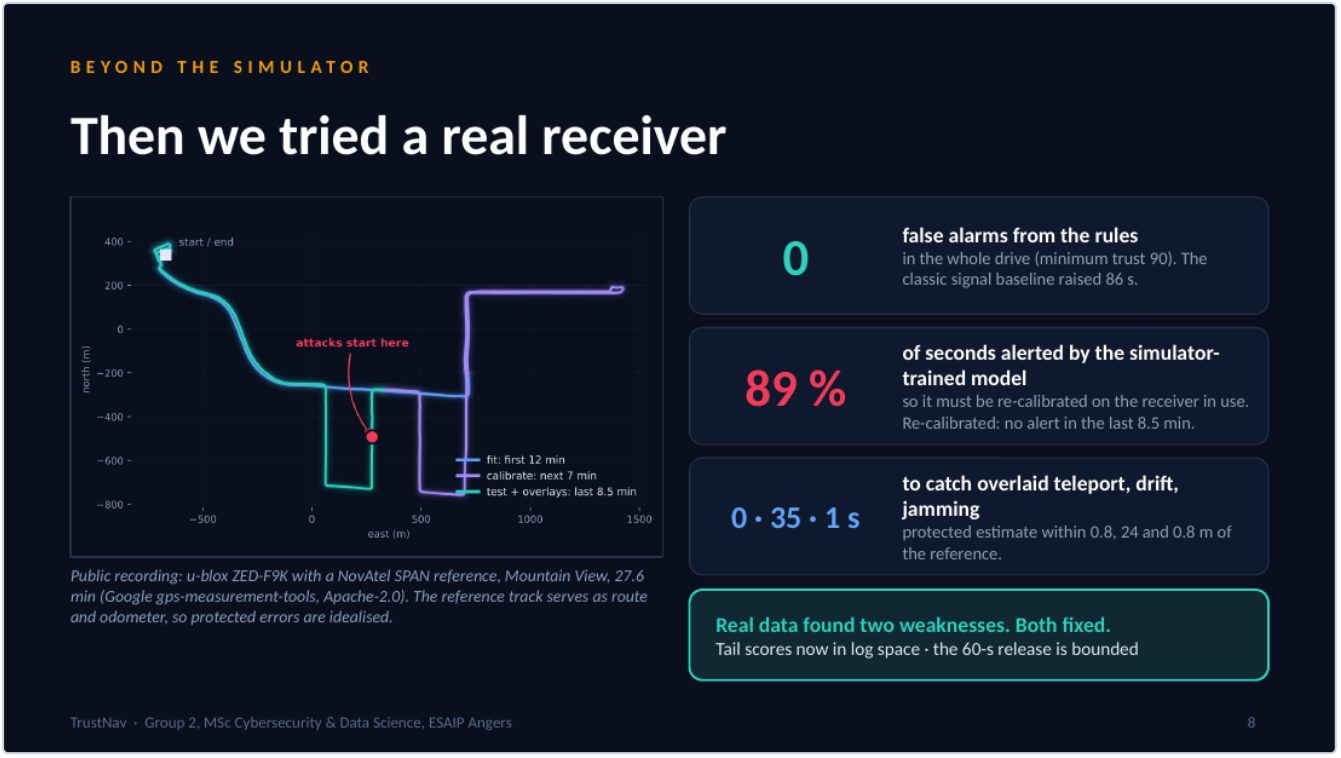
This figure shows four held-out scenarios second by second. The top row is trust: the blue line is the combined detector, the dashed and dotted lines are the rules and the model alone, the grey band is the attack, and the marker is the first alert with its delay.

Panel a, the teleport: the fix jumps 8 kilometres and is caught in the same second. Panel b is the one we are proudest of, the stealth drift: a relay-style attack that keeps the signals clean and Galileo authentication valid, moving the fix at 0.7 metres per second. No rule ever fires. The model catches it after 42 seconds, when the fix is about 12 metres off the road, where a fixed 150-metre route rule would need three and a half minutes. Panel c is jamming with recovery, caught after one second. Panel d is a drone pushed off its survey line.

The bottom row is what the response layer prevents. The orange line is the position error an ordinary screen would show; the green line is the error of the estimate TrustNav hands downstream, on a log scale. The worst reported error across the spoofing scenarios is 8 003 metres; the worst protected error is 29 metres. During the 100-second jamming outage an ordinary screen is blank; the estimate stays within 2 metres.

So: seven out of seven episodes, a median delay of one second, 8 kilometres reduced to 29 metres, and no blank screen.

8 Then we tried a real receiver



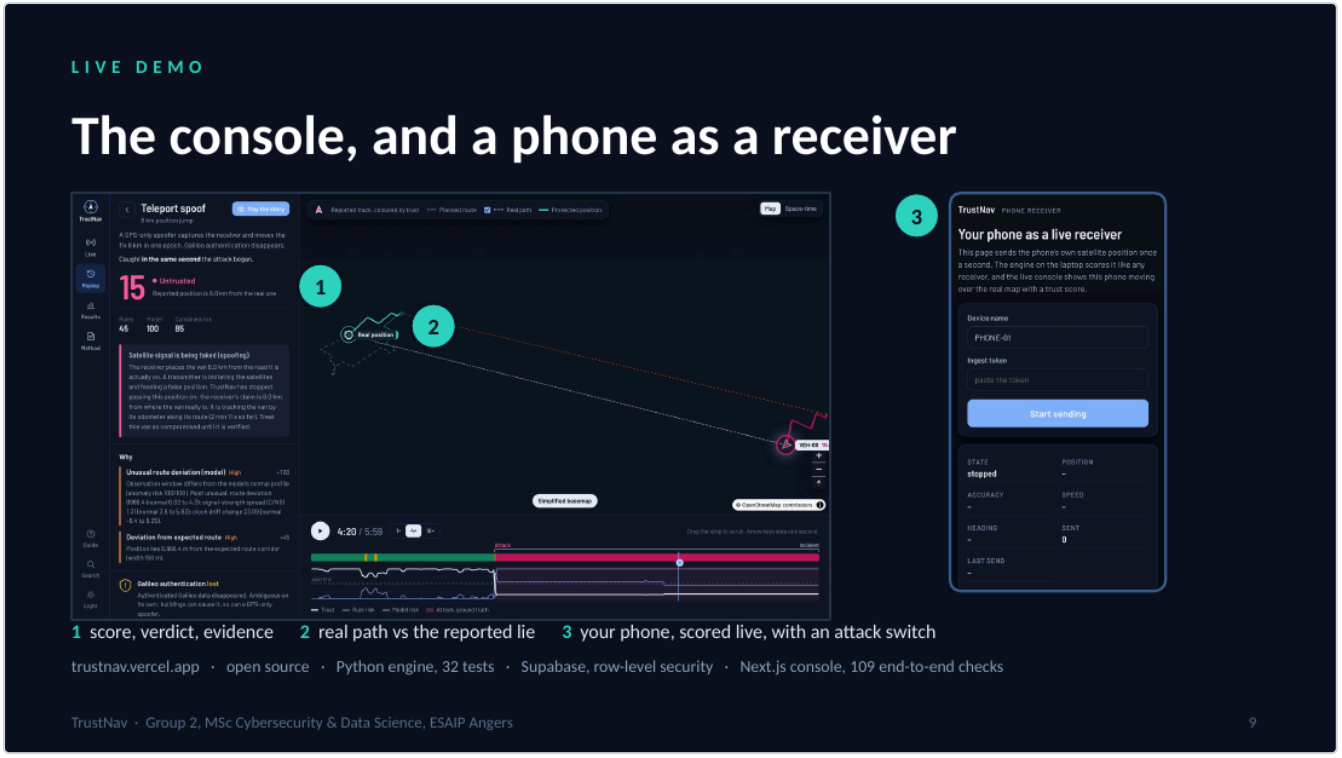
Speaker: Niranjan (1.5 min)

Everything so far came from one simulator, and we say so in the paper. So we looked for real receiver data we could use without buying hardware. Google publishes a 27.6-minute recording of a u-blox ZED-F9K driving a loop in Mountain View, with a survey-grade NovAtel SPAN reference track of the same drive. We replayed it through the same adapter and the same detector, with the reference track as the assigned route and its speed over ground standing in for the odometer.

Three results. First, the rules alone raised no false alarm in the whole drive, minimum trust 90, while the classic signal baseline raised 86 false-alarm seconds in urban canyons. Second, the model trained on the simulator alerted on 89 percent of the seconds: real receiver noise is not the simulator's noise, and this is exactly the receiver dependence the literature warns about. Re-calibrated on the first 19 minutes of the drive, it raised no alert in the remaining 8.5 minutes. Third, with the attacks of the phone page overlaid on that last segment, teleport, drift and jamming were caught after 0, 35 and 1 seconds, and the protected estimate stayed within 0.8, 24 and 0.8 metres of the reference. Those protected errors are idealised, because the route and odometer come from the reference itself.

The real data also exposed two weaknesses that we fixed: the model's tail score saturated for extreme values, so an 8-kilometre lie scored like a 2-kilometre one; and the sixty-second clean-streak release let a teleported fix that kept moving consistently back in after a minute. Both are now fixed and tested, and the simulator numbers did not change.

9 Live demo: the console, and a phone as a receiver



Speaker: Karan, with Pavan driving the console (3 min)

Now the live system at trustnav.vercel.app. Three things, briefly.

One: Replay, teleport spoof. Press play and step to the attack. The score falls from the nineties to about 12, the band turns Untrusted, and the verdict panel says in plain words that a transmitter is imitating the satellites and that the receiver places the van 8 kilometres from the road it is on. Below it, the evidence lines with the values and their normal ranges: this is what makes the decision checkable by an operator who is not a GNSS expert.

Two: the map. The reported track is coloured by trust and drawn hollow once refused; the protected position stays on the real path, with the dashed line showing the length of the lie. The response action reads ISOLATE, and the playbook lists what the dispatcher should do now.

Three: the phone. The page at /phone sends the phone's own fixes once a second; the laptop scores them like any receiver, and the phone appears on the live map with its trust score. The switch at the bottom overlays a teleport, a drift or jamming in software, nothing is transmitted, so the detection and the response can be shown on real streets. The phone gives no per-satellite signal strength, clock or OSNMA; that is what the receiver board adds later.

Under the hood: open source on GitHub, a Python engine with 32 automated tests, a Supabase back end with row-level security and a hashed ingest token, and a Next.js console with 109 end-to-end checks on every build.

If the network fails in the room, we have the replay recorded and the paper's figures; the demo degrades to the recorded scenarios.

10 What we claim, what we don't, what's next



Speaker: Pavan (1.5 min), then questions for the whole team

We want to be precise about what we claim. The evaluation numbers come from one simulator, so they are an upper bound. Route deviation assumes an assigned route, true for a delivery fleet or a survey drone, not for a free driver. The response layer is only as good as the odometer it trusts. OSNMA is simulated, and we have no multi-antenna or raw signal evidence.

On safety and ethics: nothing is transmitted at any point, every attack exists as software, and the Phase 2 hardware is receive-only. A real deployment tracks drivers, so purpose limitation, retention, three days in the demo, access control and transparency apply under the GDPR. The trust score is a research indicator, not a certified integrity figure; TrustNav is not for aviation or maritime use.

What comes next: hours of real drives in Angers, first with the phone, then with a u-blox receiver, to get a false-alarm rate per hour on our own data; replaying the Jammertest 2023 and TEXBAT recordings through a software receiver for real attack signals; a second cheap receiver against synchronous spoofing; and real OSNMA from raw Galileo pages.

In one sentence: a per-second, explained trust score and a response that keeps a lie from the systems that act on it. Thank you, we are happy to take questions.

Likely questions and short answers

Why simulate instead of using real attack data?

Transmitting attacks is illegal, and no public dataset has per-second receiver observables with routes and odometers. We simulate over real Angers geometry, keep every route in a single role, and we validated transfer on a real receiver recording, where the rules raised no false alarm. Real attack recordings (Jammertest 2023, TEXBAT) are raw RF and need a software receiver; that is the next step.

Isn't the model just learning the simulator?

Yes, and we show it: on the real receiver it alerted on 89 % of the seconds until re-calibrated. That is why the model is trained on normal data only, never on attacks, and why re-calibration on the receiver in use is part of the deployment, not an afterthought. The rules transfer as they are.

Why noisy-OR and not a classifier or an average?

Rules and model are two witnesses: either one should be able to raise the alarm. An average of 90 and 20 gives 55 and hides it; noisy-OR gives 75. A supervised classifier would learn the simulator's attacks. Noisy-OR assumes conditional independence, which jamming violates; the risk is then over-confident, which is harmless for an alert but means it should not be read as a probability.

How did you choose the thresholds and weights?

Thresholds from physics and the platform profile, not from data. The two fusion weights by grid search on validation routes only; the threshold sweep in the paper shows F1 is flat around the chosen alert line of 50, so it is not tuned to the test set.

What about false alarms in real traffic, tunnels, urban canyons?

Our normal runs include stops, cornering and urban canyons. On the held-out catalogue: 0.44 % of clean seconds, four brief episodes in 37.5 minutes, no false incident. On the real drive: zero from the rules, zero from the re-calibrated model in 8.5 minutes; we need hours of real driving to state a rate per hour with confidence, and that is the first item of the plan.

A clever spoofer could follow the road at the right speed. Would you catch it?

A spoofer that matches the route, the speed, the C/N0 spread and the clock behaviour would be hard to detect from one receiver; that is in our limitations. The odometer comparison and the route are the first defence; a second antenna on the vehicle is the next one, because one spoofer makes two real antennas report the same position.

What does the odometer add, and what if it is unavailable?

It is an input the spoofer cannot touch: it feeds a speed-mismatch rule and the dead-reckoned estimate during a hold. Without it, the hold falls back to the last trusted speed and heading, and the estimate drifts faster. On a vehicle it comes from the CAN bus or OBD-II; on the phone demo we have no odometer.

Why Galileo OSNMA and not GPS authentication?

OSNMA is the only operational civil authentication service, since 24 July 2025, and it is becoming mandatory in smart tachographs. GPS has no equivalent in service. OSNMA is one witness: it catches forged data, not a relay or jamming.

How is the system itself secured?

The browser only reads; all writes go through an edge function that checks a token stored as a SHA-256 hash; presenter controls use a bcrypt-hashed passcode with rate limiting; no secret is in the repository; row-level security on every table.

What is the false-positive cost of a hold?

A hold adds a few metres of odometer error even when the attack did not move the position (the signal-anomaly case, 20.8 m), and holds outside attacks total 26 seconds over the whole catalogue. The release rule and the ten-times bound are the trade-off between safety and availability.

What would you do with more time or budget?

About 250 euros buys a dual-band receiver board and antenna (a single-band M8/M9 class board is about 60). The plan is: record normal drives, re-calibrate, replay public attack captures, then a second receiver and real OSNMA. The code, adapters and calibration scripts for all of that already exist.

All numbers in these notes are those of the IEEE paper (trustnav.pdf, trustnav_fr.pdf) and are regenerated by paper/figures/make_figures.py and trustnav_engine.realcheck.